UGDSAI 29 · PROBLEM 9 · GROUP 9

SupportEscalator

A LangGraph support-ticket agent that resolves routine SaaS tickets automatically and pauses for a human supervisor when policy, sentiment, or unresolved work demands escalation.

Pattern: Human-in-the-Loop + Conditional Routing · PDF deck · 8 min presentation + 2 min Q&A

Team

Divyam Jindal · Bhanu Uday · Ishaan Bansal · Harman Manik

LangGraph | Pydantic v2 | Streamlit | 30/30 tests

Mosaic Cloud needs faster support without unsafe automation.

2,000+
TICKETS PER DAY
Volume from the brief

40%
CURRENT UNASSISTED
Starting point

75%
TARGET AUTO-RESOLUTION
Business goal

$200+
REFUND APPROVAL GATE
Supervisor threshold

> The risk is not just slow support. It is sending routine work to humans while risky refunds, angry customers, and unresolved bugs do not get structured human review.

The user is a support operations lead.

Jobs to be done

- Resolve routine tickets in seconds.
- Escalate risky tickets with evidence.
- Preserve an audit trail for supervisor review.
- Measure whether automation is behaving correctly.

KPIs from the brief

- Auto-resolution rate: target 75%.
- Mean time to resolution.
- CSAT by category.
- Escalation appropriateness.
- Refund exposure routed to supervisors.

One graph, four specialist paths, one escalation gate.

01 TICKET INPUT

Customer issue

Message, title, account id, requester email.

02 CLASSIFY

Choose route

Bug, billing, feature, or general.

03 SENTIMENT

Score risk

Frustration score influences escalation.

04 SOLVE

Use tools

KB search, account lookup, refund calculation.

05 ESCALATE

Pause if risky

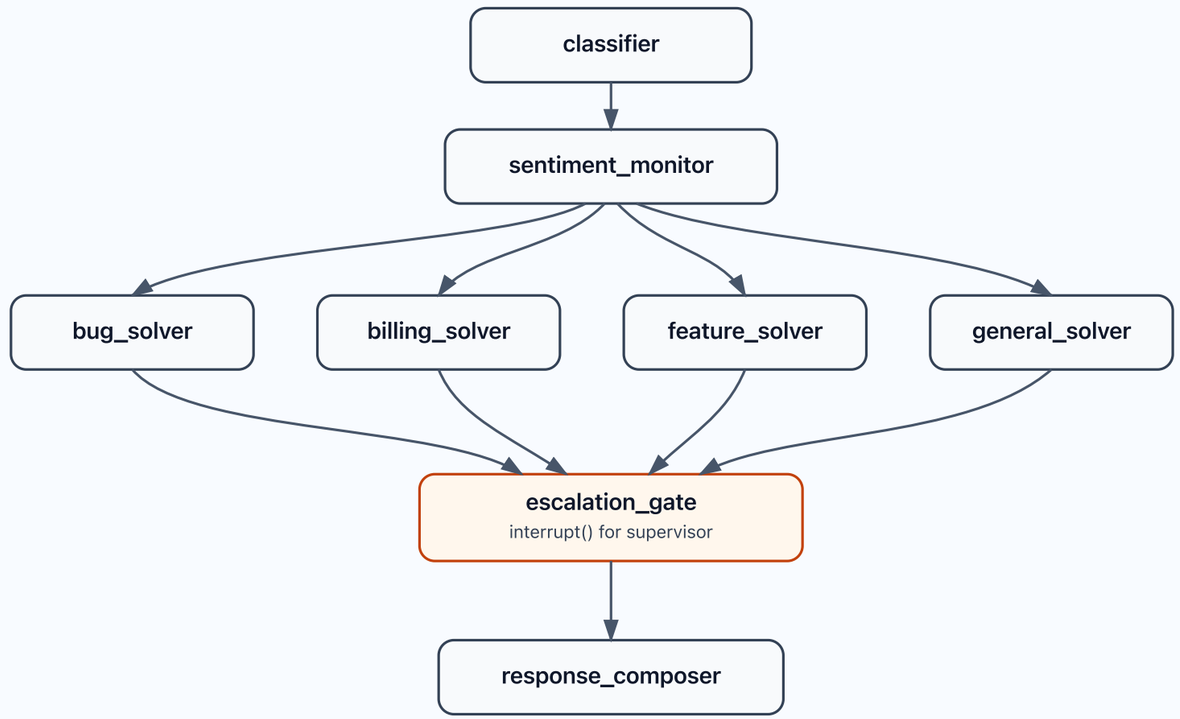
Angry tone, refund threshold, unresolved attempt.

06 RESPOND

Compose reply

Customer response plus ticket metadata.

The required graph pattern is explicit.



Committed at assets/support_escalator_graph.svg

Why this fits LangGraph

- Conditional edges route by category.
- `interrupt()` creates a real human checkpoint.
- `SqliteSaver` persists paused runs.
- Pydantic state makes transitions inspectable.

The state schema is the product contract.

STATE FIELD	WHY IT MATTERS
`ticket`	Original customer issue.
`category`	Controls the conditional route.
`resolution_attempts`	Audit trail of solver outputs.
`sentiment_score`	Risk signal for escalation.
`supervisor_input`	Human decision used on resume.
`final_response`	Customer-facing output.

Engineering proof

- OpenAI structured output for classifier and sentiment.
- Deterministic fallback when no API key is set.
- UI normalization prevents Pydantic/dict shape crashes.
- Checkpoint tests verify pause and resume behavior.

30 / 30 tests

6 evaluation cases

The demo proves all required behaviours.

1. Password reset

General ticket resolves automatically.

Route	general_solver
Escalation	None
Outcome	Final response ready

2. Duplicate billing

Refund exceeds the supervisor threshold.

Route	billing_solver
Trigger	$499 refund above $200
Outcome	Interrupt and resume

3. Angry upload bug

Tone and unresolved bug create risk.

Route	bug_solver
Trigger	Sentiment at least 0.67
Outcome	Supervisor guidance included

We tested the graph like an evaluator would.

30/30
TESTS PASSING
Latest local run

6
EVALUATION CASES
Expected behaviour checks

4
SOLVER ROUTES
Bug, billing, feature, general

3
ESCALATION TRIGGERS
Sentiment, refund, unresolved

REPRESENTATIVE CASE	EXPECTED BEHAVIOUR	EVIDENCE
Password reset	General route, no interrupt.	Graph test
Duplicate billing	Billing route, refund interrupt, resume.	Graph test
Angry upload bug	Bug route, sentiment interrupt.	Graph test
Unknown account	Unresolved billing escalates.	Evaluation suite

The UI lets evaluators exercise the pattern directly.

Customer and agent workspace

- Ticket intake and demo queue.
- Customer/account snapshot.
- Resolution timeline and solver evidence.
- Final response copy/download.

Supervisor and manager workspace

- Pending escalation reason.
- Refund exposure and account context.
- Approve or reject with guidance.
- Analytics for resolution and escalation trends.

Bonus work is real, but scoped honestly.

Implemented or documented

- LangSmith tracing setup via environment variables.
- Evaluation framework with six representative cases.
- Prompt iteration documented in README.
- Supervisor-ready escalation summary.

Next production step

- SLA timers and breach-triggered escalation.
- Zendesk sandbox adapter for ticket ingest.
- Multilingual response drafting.
- Postgres checkpointer and supervisor assignment queue.

The important fixes are now part of the system.

What broke

- `ResolutionAttempt` was a Pydantic object, not a dict.
- Memory-only state made restart demos fragile.
- External graph rendering was unreliable in sandboxed network conditions.

How we fixed it

- Added UI state normalization and regression tests.
- Switched default graph persistence to `SqliteSaver`.
- Committed a stable SVG diagram artifact.

Thank you.

The panel should remember four things: explicit LangGraph routing, real human pause and resume, 30 passing tests, and a credible support operations story. Repository, deck, architecture diagram, business memo, and demo script are committed for submission.

GitHub repository ready | 30 / 30 tests | LangGraph + Streamlit | SqliteSaver persistence

Prepared Q&A themes

- Why this LangGraph pattern instead of one chatbot?
- What happens when a node fails or a supervisor rejects?
- What minimum data is required to make this useful?
- How would this scale beyond the demo?